

End-Term Guidelines



UGDSAI 29
Designing & Deploying AI agents

Presentation Guidelines

Please find the important details for the examination as follows:

- **Weightage:** 40% of the total course Evaluation.
- **Duration:** per group 10 minutes - 8 minute presentations followed by 2 minutes of Q&A
- **Format:** PDF only | File Name: *Group_X*.
- **Group Size:** 4

Submission of Presentations :

Note: Refer to your LMS for class recordings and shared resources by the faculty over the semester.

Important Guidelines:

- The group formation for the Presentation is attached here for your reference. Please refer to the [final group list](#) and begin coordinating with your group members at the earliest.
- The attendance throughout the session is mandatory.
- The evaluation will be based on the **participation of all team members**.
- **No grades will be awarded to absentees on the presentation date**, even if their team presents. The scores of team members will not be impacted.
- Presentation duration per team: 10 minutes, followed by 2 minutes of Q&A (Please ensure effective time management, as no additional time will be allowed).

Presentation Format

- Each group will have a 10-minute presentation (8 minutes of presentation + 2 minutes of panel Q&A).
- *We will strictly follow the dates / Timings assigned.*
- If you miss the presentation, a standard 10 mark deduction applies.
- *If you miss the presentation on your respective date, we will not be able to reschedule under any circumstances.*

Submission Evaluation

You are expected to submit your presentation slides before the presentation date. Your submission deadline will be on **1st May 2026, 11:59 PM** on the LMS portal.

No submissions will be accepted post-deadline and through any other medium.

In case you submit late due to any technical issues, please drop an email, with proof to support your technical issue/glitch. If you will be unable to produce a proof document; it will result in a -5 deduction in total scores for missing the submission deadline.

All the members have to submit the files.

Important Note:

- Please ensure you are **formally dressed (Suit Up!)** for the presentation.
- A **final debrief session is compulsory** for all students.
- **No extensions will be allowed under any circumstances**, even if the presentation is incomplete.

Guidelines & Rubric



UGDSAI 29
Designing & Deploying AI agents

Guidelines & Rubric

1. Overview

The end-term examination is a group project culminating in a *10-minute presentation (8 minutes of presentation + 2 minutes of panel Q&A)*. Each group of 4 students will be assigned one of 11 problem statements and must build a working AI agent using LangGraph. This examination tests applied mastery of LangGraph patterns taught after the mid-term: multi-agent orchestration, reflection loops, human-in-the-loop interrupts, and persistent stateful workflows. Unlike the mid-term, a single tool-calling agent will not be sufficient to score well — the chosen LangGraph pattern must be implemented correctly.

2. Group Structure & Topic Assignment

- 44 students total, organised into 11 groups of 4 members each
- Each group receives one problem from the 11 problem statements
- Topic assignment method: (to be decided by instructor —Chit-Picking, first-come-first-serve, or assignment)
- No two groups may share the same problem
- All four group members must contribute visibly (see Academic Integrity section)

3. Timeline (14-day Build Window)

Day	Milestone
Day 1	Topics assigned. Groups read the brief, clarify questions, and sketch a high-level design.
Day 2-3	Architecture review: each group submits a one-page HLD (graph diagram + node responsibilities + state schema). The instructor returns feedback within 24 hours.
Day 4-9	Build phase: implement nodes, state, edges. Integrate data sources and APIs. Build the Streamlit UI.
Day 7	Optional mid-point check-in (10-minute demo to the instructor). Strongly recommended.
Day 10-12	Testing, edge cases, demo recording, slide deck.

Day 13	Code freeze. All deliverables must be submitted by the end of the day.
Day 14 onwards	Presentations during Sessions 23 and 24.

4. Mandatory Tech Stack

Non-negotiable:

- Python 3.10 or higher
- LangGraph (latest stable release)
- Pydantic v2 for all state schemas
- Git + GitHub for version control
- Streamlit for the UI (Gradio acceptable with prior approval)

LLM / API access:

- Paid API (OpenAI, Claude, or Gemini). Keys are the team's responsibility.
- No local-only / Ollama deployments (reliability risk in a live demo context)

Data & external APIs:

- Fully open - teams source their own data. CSV files, public APIs, or scraping (with rate limiting) are all acceptable.
- All data sources used must be cited in the README

AI coding tools:

- Cursor, Claude Code, Antigravity, GitHub Copilot — all allowed and encouraged
- But the team must be able to explain every line during Q&A. **Blind copy-paste will be detected and penalised.**

5. Deliverables

Submit all of the following by the end of Day 13 (24 hours before the first presentation slot), i.e., by the end of 3 May 2026:[11:59 PM]

1. GitHub repository with full working code. README must include: problem summary, setup instructions, required env vars, how to run, and architecture diagram.
2. LangGraph state diagram as PNG or SVG, generated from your graph (using `graph.get_graph().draw_png()` or equivalent). This is non-negotiable — it is the single best indicator of whether the team understands the framework.
3. **Presentation deck (PDF, 8-12 slides).**
4. Pre-recorded demo video (3 minutes maximum, screen capture with voiceover). This will play during the 'demo' segment of the presentation. Live demos are discouraged - live failures waste presentation time.
5. One-page business memo: restate the problem in your own words, identify the user persona, and list the KPIs your agent impacts.

6. Presentation Structure (8 min + 2 min Q&A)

Time	Segment	What to cover
------	---------	---------------

1 min	Problem framing	What is the business problem? Who is the user? Why does it matter? What KPIs does it impact?
2 min	Architecture walkthrough	Show the LangGraph state diagram. Walk through nodes, states, and edges. Explain why the chosen pattern was necessary.
3 min	Demo (pre-recorded)	Screen capture showing the agent in action. Must demonstrate the core pattern working (the loop, the interrupt, the multi-agent handoff, or the persistent state).
1.5 min	KPIs, edge cases, learnings	What metrics does this move? What edge cases did you handle? What broke along the way, and how did you fix it?
0.5 min	Close	What's next? Limitations. Thank the panel.
2 min	Q&A	Open questions from the panel. All four team members should be prepared to respond.

All four members must present at least one segment. The panel will direct Q&A to specific members, so everyone must understand the full system.

7. Evaluation Rubric (100 marks)

Criterion	Marks	What we look for
Functional Correctness	30	Graph runs end-to-end without errors. All required nodes are present and functional. State transitions work as designed. The demo reproduces the core behaviour described in the brief.
LangGraph Mastery	20	Pattern-specific element (cycle/interrupt / multi-agent handoff/checkpointing) is implemented correctly and justified. State is typed with Pydantic. Conditional edges are used meaningfully. Graph structure is clean.
Code Quality & Architecture	15	Clean separation of concerns. Nodes have single responsibilities. Pydantic schemas are well-designed. README is clear and complete. Reasonable error handling.
UI & UX	10	Streamlit UI is functional, handles inputs gracefully, displays meaningful output, and lets

		the evaluator exercise the key flows described in the brief.
Demo & Presentation	15	The problem is clearly framed. The architecture walkthrough is lucid. The state diagram is included and accurate. Demo runs and demonstrates the pattern. Team handles Q&A competently.
Business Thinking	10	KPIs are realistic and tied to the business problem. Edge cases and failure modes are considered. The scope is appropriate for 14 days.

8. Bonus Challenges (up to +15 marks)

Bonus marks are awarded on top of the 100-mark base, capped at a total of 115. Attempt these only after the mandatory components are complete.

- LangSmith integration for tracing and observability (+4)
- Evaluation framework: a test suite with at least 5 representative inputs and expected behaviours (+4)
- Thoughtful prompt iteration visible in commit history or README (+3)
- Problem-specific bonus items listed in each individual brief (+4)

9. Academic Integrity

- Using AI coding tools (Cursor, Claude Code, Antigravity, Copilot) is permitted and expected. They are the tools this course teaches.
- But every team member must understand every part of the codebase. Panel Q&A will include architectural questions ('why did you pick this pattern here?') and implementation questions ('walk me through what happens when X fails'). Blind copy-paste will surface quickly.
- Commit logs must show contributions from all four members. A repo with commits only from one account will result in marks redistributed to contributing members.
- Plagiarism of another group's code or of public tutorial solutions (beyond typical library usage) results in zero marks for the offending component.
- All data sources, prompt templates sourced from external repos, and any non-trivial code borrowed from tutorials must be cited in the README.

10. Submission Checklist

Before you submit, confirm all of the following:

- The GitHub repository is public or shared with the evaluator's email
- README contains: problem summary, setup steps, env vars, how to run, architecture diagram reference, data source citations
- requirements.txt or pyproject.toml is present and complete

- Graph state diagram is committed as PNG/SVG in the repo
- Demo video is uploaded (Drive, YouTube unlisted, or Vimeo) and linked in the README
- The presentation deck is submitted as a PDF
- A business memo (one page) is submitted as a PDF or Markdown
- All four team members' names and roles are listed in the README

11. Q&A Expectations

Panel questions fall into three categories. *Prepare for all three:*

Architectural:

- Why did you choose this LangGraph pattern for this problem?
- What alternative architectures did you consider and reject?
- Where are the bottlenecks in this graph if traffic scales 10x?

Implementation:

- Walk me through what happens when [specific node] fails.
- How does your state schema prevent invalid transitions?
- What happens if two workers return conflicting outputs?

Business:

- What KPI moves and by how much?
- Who in the customer's organisation would actually buy this?
- What is the minimum data this agent needs to be useful?

12. Getting Help During the 14 Days

- Dedicated office hours on Days 3, 7, and 11 (details to be shared separately)
- Course Slack/Discord channel for quick questions
- Optional Day 7 check-in: 10-minute demo to the instructor for feedback before final build
- LangGraph official docs and tutorials are strongly recommended:
<https://langchain-ai.github.io/langgraph/>

Good luck. Build something a real product team would want to deploy.

Presentation Topics



UGDSAI 29
Designing & Deploying AI agents

Presentation Topics

**11 Problem Statements across 4 LangGraph patterns • 44 students • 11 groups of 4
• 14 days build time**

1. Overview & Problem Index

This document contains 11 end-term problem statements for presentation-based assessment. Each problem requires the team to build a working AI agent using LangGraph. Problems are grouped by the LangGraph pattern they test. Groups are assigned one problem each. All problems are evaluated against a common rubric (see separate Guidelines document).

#	Title	Pattern	Industry	Difficulty
1	MarketPulse	Supervisor + Workers	E-commerce / D2C	Hard
2	DueDiligence	Supervisor + Workers	Venture Capital / Fintech	Hard
3	AgriAdvisor	Supervisor + Workers	Agri-tech	Hard
4	ContractLens	Reflection Loop	Legal-tech	Hard
5	EduTutor	Reflection Loop	Ed-tech	Hard
6	TravelPlanner	Reflection Loop	Travel	Hard
7	HealthTriage	Human-in-the-Loop	Healthcare / Telemedicine	Hard
8	PortfolioAdvisor	Human-in-the-Loop	Fintech / Wealth Management	Hard
9	SupportEscalator	Human-in-the-Loop	Customer Support / SaaS	Hard
10	RealEstateFinder	Long-Running Stateful Workflow	Real Estate / PropTech	Hard

11	JobHunter	Long-Running Stateful Workflow	HR / Careers	Hard
----	-----------	--------------------------------	--------------	------

Pattern distribution:

- Supervisor + Worker Multi-Agent — Problems 1, 2, 3
- Reflection / Self-Correction Loop — Problems 4, 5, 6
- Human-in-the-Loop (Interrupts) — Problems 7, 8, 9
- Long-Running Stateful Workflows — Problems 10, 11

Problem 1: MarketPulse — Competitive Intelligence Supervisor

Coordinate specialist workers to produce a weekly competitive brief for a D2C product team.

PATTERN	INDUSTRY	DIFFICULTY
Supervisor + Workers	E-commerce / D2C	Hard

1. Company Background

Lumen Apparel is a D2C fashion brand selling on its own Shopify store and on Myntra. Every Monday, their product manager spends six hours doing manual competitive research across five competitor brands: pulling prices, reading customer reviews, and tracking ongoing promotions.

The founding team wants to replace this Monday morning ritual with an AI agent that runs each week automatically and produces a structured competitive brief.

2. The Business Problem

Build a Supervisor agent that orchestrates three specialist Worker agents, each responsible for a different slice of the competitive landscape. The Supervisor decides which workers to invoke for a given brand, waits for all results, and synthesises a final brief.

The final output must be a one-page Markdown brief covering pricing positioning, sentiment summary, and active promotions across all tracked competitors.

3. Why LangGraph (Required Pattern)

This problem inherently needs multi-agent orchestration with shared state. A single ReAct agent would call all the tools sequentially and lose the ability to parallelise or to isolate worker failures. LangGraph lets you fan out to workers, collect their outputs in typed state, and fan back into a synthesiser node.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- Supervisor — LLM-powered router that decides which workers to dispatch based on the user query and available competitors
- pricing_worker — Extracts pricing data per SKU (mock scrape or static CSV acceptable)

- sentiment_worker — Summarises customer reviews into sentiment themes using an LLM
- promo_worker — Identifies active deals, discount depth, and promotion copy
- aggregator — Merges all worker outputs into a unified data structure
- brief_writer — Writes the final Markdown brief from the aggregated data

4.2. Graph State (Pydantic BaseModel fields):

- competitors: list[str]
- pricing_data: dict[str, list[PricePoint]]
- sentiment_data: dict[str, SentimentSummary]
- promo_data: dict[str, list[Promotion]]
- final_brief: str

4.3. Pattern-specific requirement:

At least two workers must be invoked in parallel (fan-out). The supervisor must merge their outputs in a single aggregator node. Use conditional edges from supervisor to workers based on what data is needed.

5. Mandatory Components

- Pydantic-typed GraphState with all fields above
- LLM-based supervisor (no keyword routing)
- Three distinct worker nodes with clear single responsibilities
- Fan-out to at least two workers in parallel
- Single aggregator node that synthesises worker outputs
- Streamlit UI that shows the final brief and lets the user toggle which competitors to track
- Logging of each node's input and output (print statements are fine)

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Static CSV of competitor SKUs, prices, reviews (mock)
- Shopify public catalog scraping (requests + BeautifulSoup) for real brands
- Amazon product review datasets on Kaggle for review text
- Myntra scraping with proper rate limiting

7. Expected Demo Flow

- User enters a target brand name or picks from a preset list
- Supervisor logs which workers it dispatches and why
- Each worker returns structured data, visible in the UI sidebar
- Final brief renders as formatted Markdown in the main panel
- Demo must show parallel fan-out working (logs should reveal concurrent worker execution)

8. Bonus Challenges (up to +10 marks)

- Weekly scheduler that runs the graph automatically (APScheduler or cron)
- Week-over-week delta detection in pricing
- LangSmith traces for observability
- Add a fourth worker for social listening (Twitter or Reddit)

9. Business KPIs to Track

- Time-to-insight: reduce weekly competitive scan from 6 hours to under 10 minutes
- Coverage: number of competitors and SKUs tracked per run
- Freshness: maximum age of data in the final brief
- Actionability: number of concrete recommendations per brief

Problem 2: DueDiligence — Startup Evaluation Multi-Agent

A VC analyst in a box. Four specialist workers evaluate a startup pitch across distinct dimensions.

PATTERN	INDUSTRY	DIFFICULTY
Supervisor + Workers	Venture Capital / Fintech	Hard

1. Company Background

Perpetual Ventures is an early-stage VC fund that receives 20 to 25 pitch decks per week. Each associate spends four hours on first-pass due diligence per startup before deciding whether to invite founders to a partner meeting.

The partners want a first-pass screening agent that scores each startup across four dimensions and produces a go, think, or pass recommendation with evidence.

2. The Business Problem

Build a Supervisor agent that dispatches work to four specialist workers: Financial Health, Team Background, Market Size, and Tech Moat. Each worker produces a score out of 10 with written evidence. A final synthesis node produces the investment recommendation.

The agent must accept a pitch deck (PDF or text) plus a short company metadata blob (founders' names, product category, stage) and output a structured evaluation report.

3. Why LangGraph (Required Pattern)

Each evaluation dimension is independent and benefits from a focused prompt and different tools. LangGraph lets you run the four workers in parallel, catch failures per worker, and feed structured scores into a single decision node. This is a textbook supervisor-worker pattern with a clear synthesis step.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- input_parser — Extracts structured data from the pitch deck (founders, traction, ask, stage, sector)
- Supervisor — Dispatches to the four workers based on the data present
- financials_worker — Analyses revenue, burn, runway, unit economics

- `team_worker` — Evaluates founders' backgrounds using LinkedIn or provided resume text
- `market_worker` — Sizes the TAM, SAM, and SOM using public data
- `tech_worker` — Assesses the technical moat and defensibility
- `synthesiser` — Aggregates the four scores with configurable weights into a final recommendation

4.2. Graph State (Pydantic BaseModel fields):

- `pitch_text`: str
- `company_metadata`: CompanyInfo
- `financial_score`: DimensionScore
- `team_score`: DimensionScore
- `market_score`: DimensionScore
- `tech_score`: DimensionScore
- `final_recommendation`: Literal['go', 'think', 'pass']
- `evidence_log`: list[str]

4.3. Pattern-specific requirement:

All four workers must run in parallel. The synthesiser must apply weighted scoring (weights configurable via the UI) and the final recommendation must cite evidence from each worker's output.

5. Mandatory Components

- PDF parsing for the pitch deck (pypdf or pdfplumber)
- Structured output from each worker using Pydantic
- Parallel execution of all four workers
- Configurable dimension weights in the Streamlit sidebar
- Final output shows scores, weighted total, recommendation, and per-dimension evidence
- Clear graph visualisation using `graph.get_graph().draw_png()` or similar

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Crunchbase public data for benchmarks
- Statista or public market reports for TAM estimates
- LinkedIn profiles (manual paste) for team analysis
- Sample pitch decks from Pitch Deck Hunt or failory.com

7. Expected Demo Flow

- User uploads a pitch deck PDF or pastes pitch text
- Input parser shows extracted structured data
- Four workers run in parallel with visible status
- Each worker's score and evidence appear in a dedicated UI card
- Final recommendation with reasoning shown prominently

8. Bonus Challenges (up to +10 marks)

- Comparison against three similar funded startups from a reference set
- Red-flag detection node that runs alongside workers

- Partner-style memo export as formatted PDF
- Confidence intervals per dimension

9. Business KPIs to Track

- First-pass screening time: reduce from 4 hours to under 5 minutes per deck
- Consistency: agreement rate with senior partner evaluation on a held-out set
- Coverage: percentage of deck dimensions evaluated
- Throughput: pitches evaluated per analyst per week

Problem 3: AgriAdvisor — Multi-Specialist Farm Decision Agent

Route a farmer's question to the right specialist and synthesise a prescriptive recommendation.

PATTERN	INDUSTRY	DIFFICULTY
Supervisor + Workers	Agri-tech	Hard

1. Company Background

Harvest Helper is an early-stage agri-tech startup building decision support tools for smallholder farmers. A farmer's day is full of decisions: should I irrigate today, is my crop showing disease signs, is it a good day to harvest, is the market price favourable?

Today, this information lives in four disconnected places: a weather app, an agriculture extension officer's bulletin, a pest advisory WhatsApp group, and a commodity price website. The founders want one conversational agent that answers any of these questions by routing to the right specialist.

2. The Business Problem

Build a Supervisor that classifies the farmer's question and dispatches to one or more of four specialists: Weather, Crop Suitability, Pest and Disease, and Market Price. The specialists can be invoked individually or in combination, and the final output is an actionable recommendation in plain language.

The agent must handle follow-up questions in context. Example: 'Is it a good day to spray?' should invoke the weather specialist. 'What should I plant next season?' should invoke crop suitability and market price.

3. Why LangGraph (Required Pattern)

The problem requires dynamic dispatch (which workers to run depends on the question), context retention across turns, and clean separation between specialists that use different APIs. LangGraph handles all three cleanly through conditional edges and typed state.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- Supervisor — Classifies the farmer's query and decides which workers to invoke

- `weather_worker` — Calls a weather API (Open-Meteo, IMD, or similar) and produces a relevant daily and 7-day outlook
- `crop_worker` — Recommends crop choices based on soil type, season, and water availability
- `pest_worker` — Matches described symptoms to known pests and suggests control measures
- `market_worker` — Fetches commodity prices and trend data
- `recommender` — Combines worker outputs into a single prescriptive recommendation that the farmer can act on today

4.2. Graph State (Pydantic BaseModel fields):

- `farmer_query`: str
- `farmer_context`: FarmerProfile (location, crop, acreage, soil_type)
- `invoked_workers`: list[str]
- `worker_outputs`: dict[str, Any]
- `final_recommendation`: str
- `conversation_history`: list[Message]

4.3. Pattern-specific requirement:

The supervisor must use conditional edges to dispatch to a subset of workers based on the query. The graph must handle a multi-turn conversation where the second question can invoke different workers than the first.

5. Mandatory Components

- At least one real external API call (Open-Meteo is free and works globally)
- Supervisor routing logic uses LLM classification, not keywords
- Support for three or more multi-turn questions in the same session
- Streamlit UI with a chat window and a sidebar for farmer profile
- Recommendations in plain language (not jargon), translated to Hindi, English, or Spanish (pick one)

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Open-Meteo API for weather (free, no key)
- data.gov.in for Indian agricultural market prices (Agmarknet)
- FAO crop suitability datasets
- Public plant disease image datasets (Kaggle PlantVillage)

7. Expected Demo Flow

- Farmer profile loaded in sidebar (location, crop, acreage)
- Three different questions were asked in one session, each invoking different worker combinations
- Supervisor log visible in UI showing routing decisions
- Final recommendation always includes a concrete action, not just information

8. Bonus Challenges (up to +10 marks)

- Voice input support using the browser speech API
- Image upload for pest identification (call a vision model)

- SMS-ready output format for farmers without smartphones
- Historical recommendation tracking per farmer

9. Business KPIs to Track

- Decision time: minutes saved per farm decision
- Coverage: percentage of farmer queries that get an actionable answer
- Accuracy of weather-dependent recommendations measured against outcomes
- Daily active farmers

Problem 4: ContractLens — Contract Review with Critique Loop

A drafter agent proposes redlines, a critic agent evaluates them, and they loop until the contract passes.

PATTERN	INDUSTRY	DIFFICULTY
Reflection Loop	Legal-tech	Hard

1. Company Background

Meridian Legal is a boutique law firm that reviews 50 to 80 NDAs and service agreements per month for its tech clients. Junior associates draft redlines, senior partners review and return with comments, junior associates revise. Each round costs one to two business days.

The firm wants an agent that acts as both drafter and self-critic, producing partner-quality redlines in a single run by iterating internally.

2. The Business Problem

Build a reflection-loop agent where a Drafter node proposes redlines to the contract, then a Critic node evaluates the drafter's work against a legal checklist (indemnity, liability, termination, confidentiality, governing law, data protection, IP assignment). If the critic flags issues, the graph loops back to the drafter with specific feedback. Loop terminates when the critic approves or after 3 iterations.

Final output: the marked-up contract with accepted redlines and a rationale for each change.

3. Why LangGraph (Required Pattern)

The drafter-critic pattern is impossible to implement cleanly in a linear chain because the number of iterations is data-dependent. LangGraph's conditional edges and cycles are designed exactly for this. Without LangGraph, you end up with nested while loops that are hard to trace or debug.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- parser — Loads the contract text and extracts clauses into a structured list
- . drafter — Proposes redlines. On iteration 1, it drafts cold; on later iterations, it drafts with the critic's feedback

- Critics — Evaluates the current draft against a 7-point legal checklist and returns issues with severity
- decision_gate — Conditional edge: if critic approved or max iterations reached, exit; else loop back to drafter
- formatter — Formats the approved contract with redlines and rationale

4.2. Graph State (Pydantic BaseModel fields):

- original_contract: str
- clauses: list[Clause]
- current_draft: str
- critic_feedback: list[Issue]
- iteration: int
- approved: bool
- final_output: str

4.3. Pattern-specific requirement:

The graph must contain a true cycle (drafter → critic → drafter again). The cycle must have a termination condition (approved OR iteration ≥ 3). The critic's feedback must be structured (not free text) and must actually influence the next draft.

5. Mandatory Components

- Visible iteration counter in the UI
- At each iteration, show the draft and the critic's issues side by side
- The legal checklist is configurable (seven categories minimum)
- Guard against infinite loops with a hard cap
- Final output includes a rationale for every accepted change

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- SEC EDGAR for real contract templates
- Open-source NDAs from Y Combinator SAFE docs
- Common Paper (commonpaper.com) for standard agreements
- Synthetic contracts generated by GPT for testing

7. Expected Demo Flow

- User pastes or uploads a sample contract
- Iteration 1 runs, draft and critique visible
- Iteration 2 runs with the critic's feedback threaded in
- Loop terminates, final marked-up contract shown
- Download button for the final redlined version

8. Bonus Challenges (up to +10 marks)

- Risk scoring per clause (low, Hard, high)
- Jurisdiction-specific checklists (India vs US vs EU)
- Diff view between original and final
- Human-in-the-loop override: the partner can reject the critic's verdict

9. Business KPIs to Track

- Review turnaround time: days to minutes
- Issues caught per contract (vs baseline partner review)
- Partner rework rate (how often partner overrides the agent)
- Cost per contract reviewed

Problem 5: EduTutor — Adaptive Tutor with Comprehension Retry

A tutor who teaches, checks understanding, and re-teaches with a different angle if the student is lost.

PATTERN	INDUSTRY	DIFFICULTY
Reflection Loop	Ed-tech	Hard

1. Company Background

Most AI tutors are glorified chatbots: they answer questions but don't check if the student actually understood. Hands up how many of us read a Stack Overflow answer and pretended it made sense.

Paraphrase AI, an ed-tech startup, wants a tutoring agent that detects confusion and retries the explanation using a different strategy (analogy, step-by-step, visual description) until the student gets it.

2. The Business Problem

Build a tutoring agent with a reflection loop. The Explainer explains a concept, the Check-Question Generator poses a quick check, the Student Response Evaluator grades the response and measures confidence, and a conditional edge routes back to Explainer (with a different strategy) if understanding is weak. Loop terminates on success or after 3 attempts.

The agent must track which explanation strategies have been tried and not repeat them on retry.

3. Why LangGraph (Required Pattern)

Adaptive tutoring needs conditional cycles and memory of past attempts. A linear prompt chain cannot decide at runtime whether to move forward or retry. LangGraph's state and conditional edges make this structure explicit and inspectable.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- topic_loader — Loads the concept to be taught and the student's prior level
- strategy_selector — Picks an unused explanation strategy (analogy, example, step-by-step, visual)
- explainer — Generates the explanation using the selected strategy
- question_generator — Produces a short multiple-choice or short-answer check question

- `response_evaluator` — Evaluates the student's answer and outputs a confidence score (0 to 1)
- `decision_gate` — If confidence ≥ 0.8 or attempts ≥ 3 , exit; else loop back with different strategy

4.2. Graph State (Pydantic BaseModel fields):

- `concept`: str
- `student_level`: Literal['beginner', 'intermediate', 'advanced']
- `strategies_used`: list[str]
- `current_explanation`: str
- `current_question`: str
- `student_answers`: list[str]
- `confidence`: float
- `attempts`: int

4.3. Pattern-specific requirement:

The graph must cycle at least once if the first explanation fails. The `strategy_selector` must guarantee no repeated strategy across iterations. Exit conditions: confidence threshold OR hard attempt cap.

5. Mandatory Components

- At least four distinct explanation strategies implemented
- Strategies cannot repeat within a single concept session
- Confidence scoring is explicit and visible in the UI
- Streamlit UI shows the attempt number, the strategy tried, and the running confidence
- Works for at least three concepts end-to-end (pick any three CS or math concepts)

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Khan Academy open content for concept text
- OpenStax textbooks for structured concepts
- Wikipedia article openings for topic framing

7. Expected Demo Flow

- Select a concept (e.g., 'recursion', 'Bayes theorem')
- First explanation shown, the student intentionally answers wrong
- Agent loops back with a different strategy (visible in UI)
- Student answers correctly the second time
- The summary at the end shows which strategies were tried and which worked

8. Bonus Challenges (up to +10 marks)

- Mastery tracking across multiple concepts in a session
- Adaptive difficulty of check questions
- Learner-facing explanation of why the strategy changed
- A dashboard showing which concepts most commonly trigger retries

9. Business KPIs to Track

- Mastery rate: percentage of concepts where the student reaches the confidence threshold
- Average attempts per concept
- Time to mastery per concept
- Student satisfaction (post-session rating)

Problem 6: TravelPlanner — Itinerary with Budget and Constraint Cycles

A planner that writes an itinerary, checks if it actually fits the budget and the physics of time, and replans when it doesn't.

PATTERN	INDUSTRY	DIFFICULTY
Reflection Loop	Travel	Hard

1. Company Background

Anyone who has planned a multi-city trip knows the pain: the itinerary looks great on paper, then you realise the train leaves before the museum opens, or the hotel eats 40 percent of the budget.

TripSmith wants an agent that drafts an itinerary, validates it against budget and physical constraints, and iterates until it actually works.

2. The Business Problem

Build a planner that generates a multi-day itinerary, then runs it through a Budget Validator and a Constraint Checker (transit feasibility, attraction hours, weather). If violations are found, loop back to the Planner with specific feedback until the itinerary is valid or 3 iterations are reached.

Input: destination, number of days, budget, traveller preferences. Output: a day-by-day itinerary that passes all validators.

3. Why LangGraph (Required Pattern)

Constraint satisfaction across multiple validators naturally fits the reflection-loop pattern. Separating the planner from the validators keeps prompts focused. LangGraph's state allows each validator's specific feedback to flow back into the planner's next attempt.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- input_parser — Structures user preferences, budget, dates, destination
- planner — Generates the full itinerary, using prior feedback if this is a retry
- budget_validator — Tallies all costs, compares against the budget, and returns overages per item
- constraint_checker — Verifies transit times, opening hours, and weather advisability for each day
- decision_gate — If all validators pass or iteration ≥ 3 , exit; else loop back to planner

- formatter — Produces final itinerary as a day-by-day card view

4.2. Graph State (Pydantic BaseModel fields):

- user_prefs: TravelPrefs (destination, days, budget, interests)
- current_itinerary: Itinerary
- budget_status: BudgetReport
- constraint_violations: list[Violation]
- iteration: int
- all_validators_passed: bool

4.3. Pattern-specific requirement:

The graph must cycle when validators flag issues. The planner must consume structured feedback from both validators (not just a 'try again' signal). Transit times must be validated against a real API.

5. Mandatory Components

- Real transit/distance API call (Google Maps, OSRM, or GraphHopper free tier)
- Weather validation using Open-Meteo
- Budget breakdown shown explicitly (flights, stay, food, activities)
- Structured validator outputs with per-day and per-item flags
- Demo must trigger at least one retry

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Open-Meteo for weather forecasts
- OpenStreetMap + OSRM for transit times
- Wikivoyage for destination content
- Numbeo for cost-of-living estimates

7. Expected Demo Flow

- User inputs: Paris, 4 days, \$1500 budget, interests in art and food
- Planner generates draft itinerary
- Validators flag: Louvre closed on Tuesdays and day 2 is over budget
- Planner replans using feedback
- Second pass succeeds, final itinerary shown

8. Bonus Challenges (up to +10 marks)

- Pareto optimisation: return two itineraries, one minimising cost and one maximising experience
- Weather-aware rescheduling: indoor activities on rainy days
- Multi-traveller mode with conflicting preferences
- Export to Google Calendar

9. Business KPIs to Track

- Planning time: days to minutes
- Budget adherence: percentage of itineraries that stay within budget
- Constraint violation rate in the final output

- User satisfaction post-trip

Problem 7: HealthTriage — Symptom Intake with Doctor Escalation

Conduct a structured symptom conversation, triage severity, and pause for human doctor review when red flags appear.

PATTERN	INDUSTRY	DIFFICULTY
Human-in-the-Loop	Healthcare / Telemedicine	Hard

1. Company Background

Telemedicine platforms receive thousands of symptom queries per day. Most can be handled with self-care advice. A small minority are genuine emergencies. Routing everything through doctors is wasteful; skipping doctors for emergencies is dangerous.

CarePath wants a triage agent that handles routine cases autonomously but pauses for human doctor input the moment a red flag is detected.

2. The Business Problem

Build an agent that conducts a multi-turn symptom intake, classifies urgency (LOW, HARD, HIGH, EMERGENCY), and for HIGH or EMERGENCY cases PAUSES the graph using a LangGraph interrupt and waits for a human doctor to review and approve before giving advice. LOW and MEDIUM cases proceed automatically to self-care guidance.

The pause must use LangGraph's built-in interrupt mechanism (not a hacky sleep). When the doctor inputs their decision, the graph must resume from the interrupted state.

3. Why LangGraph (Required Pattern)

LangGraph's interrupt() and checkpointner are the correct primitives for this. Without them you either need a separate queueing system or you block threads. The problem forces teams to learn interrupts, checkpointing, and graph resumption — all distinct from the midterm content.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- intake_conversation — Conducts multi-turn questioning to collect structured symptom data
- symptom_extractor — Structures collected information into a typed Symptoms object
- triage_classifier — Outputs urgency class and identifies red flags
- interrupt_gate — If urgency in (HIGH, EMERGENCY), call interrupt() and wait for doctor input
- doctor_approved_advice — Produces advice that has been reviewed by doctor
- self_care_advice — Produces advice for LOW or MEDIUM cases
- formatter — Renders final advice with appropriate disclaimers

4.2. Graph State (Pydantic BaseModel fields):

- patient_context: PatientInfo (age, sex, existing conditions)
- conversation_history: list[Message]
- symptoms: Symptoms
- urgency: Literal['LOW', 'MEDIUM', 'HIGH', 'EMERGENCY']
- red_flags: list[str]
- doctor_decision: Optional[DoctorDecision]
- final_advice: str

4.3. Pattern-specific requirement:

Must use LangGraph's interrupt() or equivalent checkpoint mechanism. The graph must pause, be resumable, and actually integrate the doctor's input into the final advice (not just log it and proceed).

5. Mandatory Components

- Use LangGraph checkpointer (MemorySaver or SqliteSaver)
- interrupt() in the interrupt_gate node
- Streamlit UI with two modes: patient view and doctor view
- Doctor view shows the triage context and lets them approve, modify, or reject the auto-advice
- Clear medical disclaimer in final output
- Red flag list is documented and visible

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- NHS symptom checker content (public)
- Mayo Clinic public symptom database
- CDC red flag guidelines
- Synthetic patient histories

7. Expected Demo Flow

- Patient 1 (LOW urgency headache): agent handles entirely autonomously
- Patient 2 (chest pain in 55-year-old): agent triages as EMERGENCY, graph pauses
- Demo switches to doctor view, doctor approves with modification
- Graph resumes, final advice incorporates doctor's input
- Show the checkpoint log proving the state was persisted

8. Bonus Challenges (up to +10 marks)

- Doctor-side dashboard showing all paused cases in a queue
- Timeout behaviour: if no doctor response in X minutes, escalate
- Patient history aware triage (conditions lower thresholds)
- Multi-language intake

9. Business KPIs to Track

- Doctor time saved per shift
- Red flag miss rate (critical — must be zero on a held-out set)
- Average time-to-advice for LOW/MEDIUM vs HIGH cases

- Doctor override rate on auto-advice

Problem 8: PortfolioAdvisor — Investment Rebalancer with Trade Approval Gates

Analyse portfolio drift, propose trades, pause for human approval on anything material.

PATTERN	INDUSTRY	DIFFICULTY
Human-in-the-Loop	Fintech / Wealth Management	Hard

1. Company Background

Robo-advisors auto-rebalance portfolios to stay aligned with target allocations. Regulators in most jurisdictions require user approval for trades above specific thresholds. Current implementations either pause for every trade (annoying) or skip approvals (non-compliant).

LedgerLine wants an agent that auto-executes small rebalancing trades but pauses for user approval on any trade above \$10k or any trade in a volatile asset class.

2. The Business Problem

Build an agent that loads a portfolio, computes drift from target allocation, proposes rebalancing trades, and for each trade above threshold **PAUSES** for user approval using LangGraph interrupts. Approved trades are executed (simulated); rejected trades are skipped. The final report summarises what was done and why.

Thresholds are configurable: default \$10k per trade OR any trade in assets with 30-day volatility > 20 percent.

3. Why LangGraph (Required Pattern)

Each approval-pending trade is a separate interrupt point. Managing multiple interrupts with shared state is exactly what LangGraph is built for. A naive implementation would need a state machine library, a queue plus a database. LangGraph gives you all three.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- `portfolio_loader` — Loads current holdings and target allocation
- `drift_analyser` — Computes percentage drift per asset and flags those that need rebalancing
- `trade_proposer` — Generates a list of proposed buy/sell trades with rationale
- `approval_gate` — For trades above threshold, interrupts; for others, auto-approves
- `executor` — Simulates execution of approved trades and updates portfolio
- `report_generator` — Produces a final report of all proposed, approved, rejected, and executed trades

4.2. Graph State (Pydantic BaseModel fields):

- portfolio: dict[str, Holding]
- target_allocation: dict[str, float]
- drift: dict[str, float]
- proposed_trades: list[Trade]
- approved_trades: list[Trade]
- rejected_trades: list[Trade]
- executed_trades: list[Trade]
- final_portfolio: dict[str, Holding]

4.3. Pattern-specific requirement:

Must handle multiple sequential interrupts (one per trade above threshold), not just a single pause. Each interrupt must carry enough context for the user to make an informed decision (trade size, rationale, expected impact).

5. Mandatory Components

- LangGraph checkpointing with resumable state
- Multiple interrupts per run (at least two in demo)
- Volatility computation uses real historical prices (yfinance is free)
- Streamlit UI with a trade-by-trade approval card
- Final report exportable as PDF or CSV

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- yfinance for historical prices and volatility
- Alpha Vantage free tier for fundamentals
- Sample portfolios (CSV of tickers and weights)

7. Expected Demo Flow

- Load a portfolio with clear drift (one asset 40% over target)
- Agent proposes 5 trades; 2 are above threshold
- Demo pauses twice, user approves one and rejects one
- Executor simulates the approved trade
- Final report shows all 5 proposed trades with final status

8. Bonus Challenges (up to +10 marks)

- Tax-loss harvesting logic
- Scenario preview: what the portfolio looks like after all proposed trades vs after approved-only
- Multi-account support (spouse, kids, retirement)
- Compliance log exportable in a regulator-friendly format

9. Business KPIs to Track

- Compliance rate (percentage of material trades with user approval)
- User friction (approvals per quarter)
- Drift-correction effectiveness
- Rebalancing cost as a percentage of AUM

Problem 9: SupportEscalator — Ticket Resolution with Conditional Escalation

Resolve routine support tickets automatically; escalate to a human supervisor when policy or sentiment demands it.

PATTERN	INDUSTRY	DIFFICULTY
Human-in-the-Loop + Conditional Routing	Customer Support / SaaS	Hard

1. Company Background

Mosaic Cloud handles 2000+ support tickets per day. Most are routine: password resets, billing clarifications, feature questions. A minority need engineer escalation (bugs), account manager attention (billing disputes over \$500), or supervisor approval (refunds over \$200).

Current first-line support resolves about 40 percent unassisted. The company wants an agent that can resolve 75 percent autonomously and escalate the rest cleanly.

2. The Business Problem

Build an agent that classifies incoming tickets by category (bug, billing, feature, general), routes to a category-specific solver node, attempts resolution using tools (KB search, account lookup, refund calculator), and PAUSES for human supervisor when escalation criteria are met (angry tone detected, refund over threshold, unresolved after tool use).

Final output: a response to the customer that either resolves their issue or explains the escalation path, plus a ticket metadata update.

3. Why LangGraph (Required Pattern)

This problem combines two LangGraph strengths: conditional routing to category-specific subgraphs AND interrupt-based escalation. Building both in one graph teaches the combination, which is the most common real-world pattern.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- classifier — Classifies ticket category
- router — Conditional edges to category-specific solver
- bug_solver — Searches known issues, requests reproduction steps if needed
- billing_solver — Looks up account, explains charges, computes refunds
- general_solver — Searches FAQ / KB
- sentiment_monitor — Runs in parallel, flags anger or frustration signals
- escalation_gate — Interrupts for supervisor if criteria met
- response_composer — Composes final customer reply

4.2. Graph State (Pydantic BaseModel fields):

- ticket: TicketInput

- category: Literal['bug', 'billing', 'feature', 'general']
- resolution_attempts: list[ResolutionAttempt]
- sentiment_score: float
- escalation_reason: Optional[str]
- supervisor_input: Optional[str]
- final_response: str

4.3. *Pattern-specific requirement:*

Graph must combine conditional routing (to category-specific paths) with interrupt-based escalation. Sentiment monitor can be parallel or sequential but must influence the escalation gate.

5. **Mandatory Components**

- At least three category-specific solver paths
- Working interrupt for supervisor escalation
- KB / FAQ built from a provided or open dataset
- Sentiment detection (can use a simple LLM prompt)
- Streamlit UI with separate customer and supervisor views
- At least one ticket in the demo goes through full escalation

6. **Suggested Data / APIs**

Teams are free to source their own data. Suggested starting points:

- Public SaaS FAQ pages (scrape a few)
- Sample ticket datasets on Kaggle (Customer Support on Twitter, etc.)
- Mock account database (CSV)

7. **Expected Demo Flow**

- Ticket 1: 'how do I reset my password?' — general_solver resolves it
- Ticket 2: 'I was charged twice' — billing_solver resolves with a refund
- Ticket 3: 'this is my third email, and no one is helping!!!' — sentiment flagged, escalation to supervisor
- Demo switches to supervisor view, supervisor provides guidance
- Final response composed with supervisor input

8. **Bonus Challenges (up to +10 marks)**

- SLA tracking (response time budget per category)
- Auto-generated ticket summary for the supervisor
- Integration with a real help desk tool (Zendesk sandbox)
- Multi-language ticket support

9. **Business KPIs to Track**

- Auto-resolution rate (target 75%)
- Mean time to resolution
- Customer satisfaction (CSAT) per category
- Escalation appropriateness (supervisor agreement with escalation trigger)

Problem 10: RealEstateFinder — Persistent Buyer Preference Learning

An agent that remembers a buyer's evolving preferences across weeks of property search.
PATTERN

PATTERN	INDUSTRY	DIFFICULTY
Long-Running Stateful Workflow	Real Estate / PropTech	Hard

1. Company Background

Home buying is iterative. A buyer tours properties over weeks or months, their preferences evolve, and they realise they care more about natural light than square footage after seeing a dark apartment. Today's property portals don't learn — they just filter on fixed criteria.

NestSage wants an agent that maintains a persistent preference state across sessions, shows new matching listings each time, receives buyer feedback, and updates preferences accordingly.

2. The Business Problem

Build an agent with a persistent state that survives across sessions using LangGraph's checkpointer. Each session: load previous state, fetch new listings, match against learned preferences, present top 5, receive thumbs up/down with comments, update preference weights, save state for next session.

The demo must show state persisting: session 1 cold-starts with generic preferences, session 2 picks up where session 1 left off, and shows different listings based on past feedback.

3. Why LangGraph (Required Pattern)

Persistent checkpointing is a first-class feature of LangGraph, not an afterthought. Teams will learn SqliteSaver / Postgres checkpointer, serialisation of complex state, and graph resumption. None of these are covered in a linear LangChain chain.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- `state_loader` — Loads checkpointed state from previous sessions (or initialises cold)
- `listing_fetcher` — Fetches new listings matching broad criteria (city, price range)
- `matcher` — Scores each listing against current preference weights
- `Ranker` — Picks top 5
- `presenter` — Formats for UI display
- `feedback_receiver` — Captures buyer thumbs up/down and free-text comments
- `preference_updater` — Uses LLM to parse feedback into preference weight adjustments
- `state_saver` — Checkpoints the updated state

4.2. Graph State (Pydantic BaseModel fields):

- buyer_profile: BuyerProfile (budget, city, hard_requirements)
- preference_weights: dict[str, float]
- seen_listings: list[str]
- feedback_log: list[FeedbackEvent]
- session_count: int
- last_updated: datetime

4.3. Pattern-specific requirement:

Must use SqliteSaver or equivalent persistent checkpointer (not MemorySaver). State must survive script restart. Demo must prove it: run session 1, stop the app, restart, run session 2, show session 2 remembers session 1.

5. Mandatory Components

- Persistent checkpointer (SQLite recommended for 14-day scope)
- The preference model has at least 6 dimensions (price, size, location, light, age, amenities)
- Feedback parsing uses LLM to infer weight deltas, not keyword matching
- Streamlit UI with a clear session counter and a 'preferences learned so far' panel
- Demo with 3+ sessions showing preference drift

6. Suggested Data / APIs

Teams are free to source their own data. Suggested starting points:

- Kaggle real estate datasets (NYC, LA, Bengaluru)
- Zillow public API (limited, register for key)
- 99acres / MagicBricks scraping (India)
- Synthetic listings generated by LLM

7. Expected Demo Flow

- Session 1: buyer sees generic top 5, thumbs down 3 for being too dark
- Script restarts (shown on camera)
- Session 2: agent shows different listings prioritising natural-light keywords, preferences panel shows new weight on 'light.'
- Session 3: buyer thumbs-up 4 of 5, agent refines further

8. Bonus Challenges (up to +10 marks)

- Negotiation aide: estimated fair price based on comparables
- Tour scheduling integration (Google Calendar)
- Explanation mode: 'I showed you this because you liked X.'
- Multi-buyer mode (couple with differing preferences)

9. Business KPIs to Track

- Preference inference accuracy (blind test against stated preferences)
- Sessions to first 'strong yes' from buyer
- Listings filtered out as a percentage of cold-start results
- Buyer engagement (sessions per week)

Problem 11: JobHunter — Persistent Candidate Search Assistant

A job-hunting copilot that tracks applications, learns from rejections, and refines its search over weeks.

PATTERN	INDUSTRY	DIFFICULTY
Long-Running Stateful Workflow	HR / Careers	Hard

1. Company Background

Active job seekers apply to 50 to 100 roles over a month. They manage applications in spreadsheets, tailor resumes manually, and forget which feedback applies to which role. Today's job boards show listings but do not learn from a candidate's evolving outcomes.

Pathmark wants an agent that maintains the candidate's full search state — profile, applications, outcomes — and adapts its recommendations as feedback comes in.

2. The Business Problem

Build a stateful agent that, on each session, fetches new matching jobs, ranks them against the candidate's evolving profile, tracks new applications the candidate logs, parses feedback from rejections and offers, and updates the search strategy.

The agent must detect patterns in rejections (e.g., 'failing at system design rounds') and surface specific preparation resources or adjust the seniority of the roles it recommends.

3. Why LangGraph (Required Pattern)

This problem exercises persistent state, pattern detection over historical data, and strategy adaptation — all requiring more than a single prompt. LangGraph's checkpointing plus conditional logic around the historical state is the right abstraction.

4. Required Graph Architecture

4.1. Minimum nodes (build all of these):

- `state_loader` — Loads candidate profile, application history, feedback log
- `job_fetcher` — Fetches new roles matching the current strategy
- `matcher` — Scores each role against profile and learned preferences
- `ranker` — Applies evolving fit-score logic, returns top 10
- `application_tracker` — Records new applications logged by the candidate
- `feedback_parser` — Processes candidate-entered rejection/offer / interview feedback
- `pattern_detector` — Detects recurring themes in rejections
- `strategy_updater` — Adjusts seniority level, role type, or prep recommendations based on patterns
- `state_saver` — Checkpoints everything for next session

4.2. Graph State (Pydantic BaseModel fields):

- candidate_profile: CandidateProfile
- applications: list[Application] (with status: applied, rejected, interviewing, offer)
- feedback_log: list[FeedbackEvent]
- rejection_patterns: list[Pattern]
- current_strategy: SearchStrategy
- sessions_count: int

4.3. Pattern-specific requirement:

Persistent checkpointer required. Pattern detection must examine at least 3 historical events before triggering a strategy change. Strategy changes must be explicit and explainable in the UI.

5. Mandatory Components

- SqliteSaver or Postgres checkpointer
- At least 5 typed application statuses (applied, screening, interviewing, rejected, offer)
- Pattern detection over rejection feedback (keyword-based minimum, LLM-based for bonus)
- Strategy update must change at least one search parameter visibly
- Streamlit UI with three panels: jobs, applications log, strategy/patterns
- Demo across 3+ sessions with evolving state

6. Suggested Data / APIs

- Teams are free to source their own data. Suggested starting points:
- LinkedIn Jobs (manual copy-paste acceptable)
- Greenhouse / Lever public job boards
- Kaggle job posting datasets for initial population
- Candidate resume as text file input

7. Expected Demo Flow

- Session 1: cold start, agent recommends 10 roles based on resume
- Candidate logs 5 applications + 2 rejections with feedback
- Session 2: agent shows the detected pattern (e.g., 'system design weakness'), recommends specific prep, and adjusts seniority down
- Session 3: candidate shows improvement, agent returns seniority to target

8. Bonus Challenges (up to +10 marks)

- Resume-tailoring suggestions per role
- Cover letter drafts per application
- Calendar integration for interview tracking
- Salary negotiation assistant when offers arrive

9. Business KPIs to Track

- Interview conversion rate (applications to interviews)
- Offer rate (interviews to offers)
- Time from application to outcome per role
- Candidate-reported usefulness of pattern detection